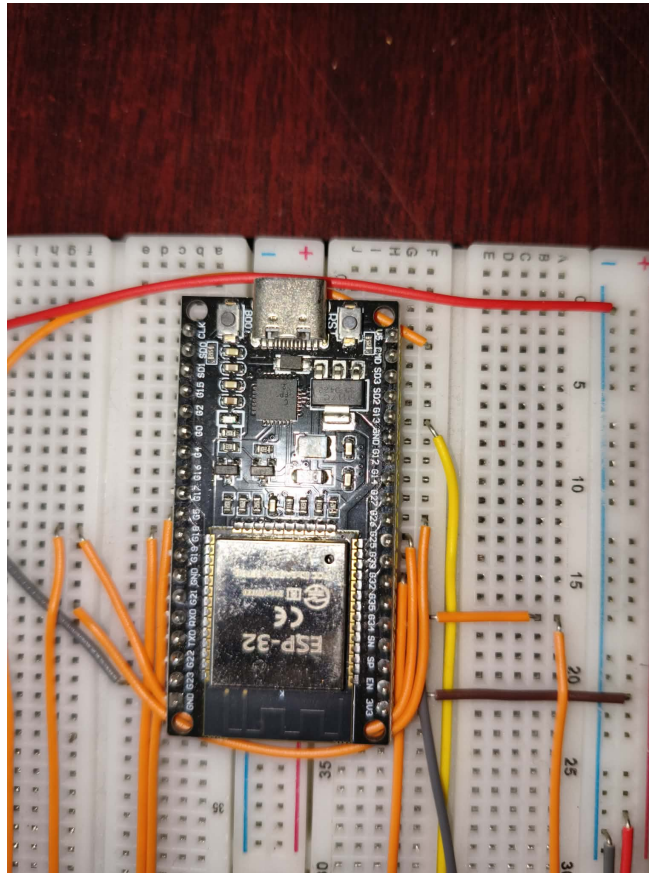


AquaGuard IoT

Hardware Wiring & Arduino IDE Setup Guide

This guide covers two things needed to bring the AquaGuard pond-monitoring hardware back up after it was taken apart for transport: (1) setting up the Arduino IDE from a completely fresh install so it can program the ESP32 board, and (2) reconnecting every sensor and actuator one at a time, with a quick test after each one, so a wiring mistake is caught immediately instead of after everything is buttoned up.

This is written assuming no prior Arduino/ESP32 experience — every step is a literal action to take, not a summary.



The ESP32 dev board (USB-C, 30-pin “WROOM-32” style) as mounted on the breadboard for this project. Pin names are printed directly on the board next to each pin — that silkscreen text is the source of truth for every wiring step in this guide.

Part 1 — Setting up the Arduino IDE from scratch

Do this once, on whichever computer will be used to write/upload code to the ESP32. If the IDE is already installed and has worked with this project before, skip to Part 2.

Step 1.1 — Install the Arduino IDE

- Go to **arduino.cc/en/software** and download the Arduino IDE 2.x installer for your operating system (Windows / Mac / Linux).
- Run the installer and accept the defaults. On Windows, also accept the driver installation prompts that appear partway through — these are the USB-to-serial drivers the ESP32 board needs.
- Open the Arduino IDE once to confirm it launches.

Step 1.2 — Add ESP32 board support

The Arduino IDE only knows about official Arduino boards by default. ESP32 support has to be added manually, once:

- Open **File** → **Preferences** (Windows/Linux) or **Arduino IDE** → **Settings** (Mac).
- Find the field labeled “**Additional Boards Manager URLs**” and paste this URL into it (add it on its own line if the field already has something in it):

```
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
```

Click **OK** to save.

- Open **Tools** → **Board** → **Boards Manager...**
- Search for “**esp32**”. Install the package published by **Espressif Systems** (this can take a few minutes — it downloads compilers and tools, not just board definitions).

Step 1.3 — Select the board and port

- Plug the ESP32 into the computer via USB-C.
- In the Arduino IDE, open **Tools** → **Board** → **esp32** and choose “**ESP32 Dev Module**” (this generic option matches the 30-pin WROOM-32 board used in this project).
- Open **Tools** → **Port** and select the port that appears when the board is plugged in (Windows: something like **COM3**, **COM5**; Mac: **/dev/cu.usbserial-...** or **/dev/cu.wchusbserial-...**; Linux: **/dev/ttyUSB0**).

■ *If no port shows up at all: the USB cable may be a charge-only cable (no data lines) — try a different cable first. If a port still doesn't appear, the board's USB-to-serial chip driver (commonly CP2102 or CH340) may need a manual driver install — search “CP210x driver” or “CH340 driver” for your OS.*

Step 1.4 — Install the two libraries this project needs

Open **Sketch** → **Include Library** → **Manage Libraries...** and install:

Library name to search	Author	Why it's needed
FirebaseESP32	Mobizt	Talks to the Firebase Realtime Database (sensor upload, pump/servo control)
ESP32Servo	Kevin Harrington / Jim Studt	Drives the servo motor from an ESP32 pin (the standard Arduino Servo library doesn't support ESP32)

(**WiFi.h** and **math.h**, also used in the code, come bundled with the ESP32 board package from Step 1.2 — nothing extra to install for those.)

Step 1.5 — Prove the toolchain works before touching sensors

- Open **File** → **Examples** → **01.Basics** → **Blink**.
- Click the **Upload** button (right-pointing arrow icon). Watch the bottom log — it should end with “**Hard resetting via RTS pin...**” and no red error text.
- The onboard LED should start blinking. If this works, the IDE, drivers, board selection, and port are all correct — any problems from here on are wiring or code, not toolchain setup.

■ If upload fails with a timeout (“Failed to connect to ESP32: Timed out waiting for packet”): hold the **BOOT** button on the board (see photo, Part 1 photo above) while the upload starts, release it once you see “Connecting...” dots appear, then let it finish. Some ESP32 boards need this manually; others auto-reset.

Part 2 — ESP32 pin reference for this project

Every pin name below (e.g. “G5”, “G18”) is printed directly on the board next to its pin — match the text on your physical board, not the position in this table, since layout can vary slightly between board batches.

Code constant	GPIO / pin	Used for	Why this pin
TRIG_PIN	GPIO 5	Ultrasonic trigger (water level)	Digital output, no restrictions
ECHO_PIN	GPIO 18	Ultrasonic echo (water level)	Digital input, no restrictions
TDS_PIN	GPIO 35	TDS / water-quality sensor	ADC1 pin — stays accurate with WiFi on
THERM_PIN	GPIO 32	Thermistor (temperature)	ADC1 pin — stays accurate with WiFi on
PH_PIN	GPIO 34	pH sensor	ADC1 pin — stays accurate with WiFi on
PUMP1_PIN	GPIO 25	Relay 1 (pump / motor 1)	Digital output
PUMP2_PIN	GPIO 26	Relay 2 (pump / motor 2)	Digital output
SERVO_PIN	GPIO 13	Servo motor	Supports the PWM signal a servo needs
—	3V3	Power for sensor modules	Regulated 3.3V — matches ESP32 logic level
—	5V / VIN	Power for ultrasonic sensor, relay module	These specific modules are rated for 5V
—	GND (any)	Common ground — every module connects here	All modules must share one ground with the ESP32

■ **GPIO 6–11** (often printed as **CLK / SD0 / SD1 / SD2 / SD3 / CMD** on this board) connect internally to the board's flash memory chip. Never wire anything to these six pins — using them will make the board fail to boot.

Logic level: ESP32 GPIO pins read/write 3.3V, not 5V. The ultrasonic sensor's ECHO output is 5V — see the voltage-divider note in Part 3, Step 3.2.

Common ground rule: every sensor's GND and every relay/pump's GND must connect back to the ESP32's GND — even parts powered from a separate battery or supply. This is the single most common cause of “worked alone, broke when everything was connected” problems.

Part 3 — Reconnecting the hardware, one part at a time

Wire and test each part in this order before moving to the next — cheapest/simplest first, and the current-draw / power-supply parts (pumps, servo) last, so a mistake there can't take down something that was already confirmed working.

Step 3.1 — ESP32 alone: WiFi + Firebase sanity check

Before wiring any sensor, upload the project code as-is (sensors will just read garbage/zero, that's expected for now). Open the Arduino IDE's **Serial Monitor** (top-right icon, or **Tools** → **Serial Monitor**) and set the baud rate dropdown in its corner to **115200** — this must match the code's **Serial.begin(115200)** or the output will show garbled text.

You should see:

```
Connecting to WiFi.....
WiFi Connected!

Aquaculture Sensor Suite V1 Online
```

If this doesn't appear: check **WIFI_SSID** / **WIFI_PASSWORD** in the code match whatever network is actually available right now (a phone hotspot name/password may have changed since the hardware was last run), and that **FIREBASE_HOST** / **FIREBASE_AUTH** still match the Firebase project. Don't wire a single sensor until this step works — every later step depends on it.

Step 3.2 — Ultrasonic sensor (HC-SR04) → water level

HC-SR04 pin	Connects to
VCC	ESP32 5V (this sensor needs 5V, not 3.3V, to work reliably)
GND	ESP32 GND
TRIG	ESP32 GPIO 5
ECHO	ESP32 GPIO 18 — through a voltage divider, see below

■ The HC-SR04's ECHO pin outputs a 5V pulse, but ESP32 GPIOs only tolerate 3.3V. Put a 1kΩ resistor in series between ECHO and GPIO18, then a 2kΩ resistor from that same GPIO18 point down to GND. This voltage-divider drops the 5V pulse to roughly 3.3V before it reaches the ESP32.

Test: re-upload, open Serial Monitor, wave a hand in front of the sensor at different distances. The **Distance** : line should change accordingly. Don't move on until this is stable.

Step 3.3 — Thermistor (temperature)

This is a plain voltage-divider circuit — just the thermistor and one resistor, no separate module board:

```
3.3V ■■[ NTC Thermistor ]■■ GPIO32 ■■[ 4.7kΩ resistor ]■■ GND
```

- One leg of the thermistor → ESP32 **3.3V**

- Other leg of the thermistor → **GPIO 32**, and also → one leg of the 4.7kΩ resistor
- Other leg of the 4.7kΩ resistor → **GND**

GPIO32 sits at the junction between the thermistor and resistor — that junction voltage is what the code's temperature formula is built around. Swapping which side the thermistor/resistor sit on will invert the reading (goes down when it should go up).

Test: re-upload, check the Serial Monitor's **Temp :** line reads a sane room temperature (roughly 20–30°C). Hold the thermistor between two fingers — it should climb within a few seconds.

Step 3.4 — TDS sensor (water quality / dissolved solids)

TDS sensor kits come as a probe wired into a small signal-conditioning module board — the module already outputs a clean analog voltage, no separate divider circuit needed:

TDS module pin	Connects to
VCC	ESP32 3.3V (check your module's silkscreen; if it supports 3.3–5V, use 3.3V since GPIO35 can only safely read up to 3.3V)
GND	ESP32 GND
Signal / AOUT	ESP32 GPIO 35

The probe (two metal prongs) plugs into the module board via its screw terminal or JST connector — that part doesn't touch the ESP32 directly.

Test: with the probe dry (in air) expect a low/noisy **TDS :** value. Dip it in a glass of tap water — the number should rise and settle within the ~30 readings the code averages over.

Step 3.5 — pH sensor

Same pattern as the TDS sensor — a probe plus a signal-conditioning module:

pH module pin	Connects to
VCC	ESP32 3.3V (check the module's rating first — some pH modules are 5V-only; if so, power from 5V but confirm the analog output itself doesn't exceed 3.3V, or add a divider like Step 3.2)
GND	ESP32 GND
PO / Signal	ESP32 GPIO 34

Calibration: the code has **ph_offset = 0.0**. Once wired, dip the probe in a known-pH liquid (a pH 7 buffer solution if available, or clean tap water as a rough approximation) and see what the Serial Monitor reports. Adjust **ph_offset** up or down until the reading matches — the raw formula is only a rough default, this calibration step is not optional.

Test: confirm **pH :** shows a plausible number (0–14) and moves when the probe is switched between plain water and something acidic (e.g. diluted vinegar).

Step 3.6 — Relay module + 2 pumps (“2 motors”)

This is the first step involving real current draw — go carefully. The relay’s control side connects to the ESP32; the pumps do **not**.

Relay module pin	Connects to
VCC	ESP32 5V (most relay modules need 5V to reliably switch, even with a 3.3V trigger signal)
GND	ESP32 GND
IN1	ESP32 GPIO 25
IN2	ESP32 GPIO 26

The pumps connect through the relay’s **other** side (screw terminals marked COM / NO / NC), wired into the pump’s own power supply circuit:

```
Pump power supply (+) → Relay COM
Relay NO                → Pump (+)
Pump (–)                → Pump power supply (–)
```

Use **NO** (Normally Open) so each pump is OFF by default and only switches on when the relay activates — matching the code’s startup state (**digitalWrite(PUMP1_PIN, HIGH) = OFF**).

The code drives these pins **active LOW** (GPIO LOW = pump ON). Most hobbyist relay modules default to this behavior, but check your specific module’s markings — a few are active-HIGH, which would make the pump logic backwards until the code or the module’s jumper is changed.

■ *Never power the pump motors directly from the ESP32’s 5V or 3.3V pins — motors draw far more current than the ESP32 can safely supply. They need their own external power source, sharing only a common GND with the ESP32.*

Test: in the Firebase Realtime Database console, manually set **/pumps/pump1** to **true**. The relay should click and its indicator LED should light. Confirm both relays click independently before connecting real pumps to the output side.

Step 3.7 — Servo (water-diversion mechanism)

Servo wire	Connects to
Signal (orange/yellow)	ESP32 GPIO 13
VCC (red)	ESP32 5V — see warning below
GND (brown/black)	ESP32 GND

■ *If the servo is anything beyond a tiny “micro” servo (SG90-class), powering it from the ESP32’s own 5V pin can brown out the board when the servo moves — this shows up as random resets or WiFi dropping right when the servo triggers. If that happens, move the servo’s VCC to the same external supply as the pumps instead (still sharing GND with the ESP32).*

Test: in the Firebase console, set **/servoTrigger** to **true**. The servo should sweep to 180°, hold for 3 seconds, return to 0°, and reset the Firebase flag back to **false** on its own.

Step 3.8 — Full integration check

With everything wired, power up and confirm in the Serial Monitor that all four sensor lines print sane values at the same time, then check the Firebase Realtime Database console shows **/sensor/temp**, **/sensor/tds**, **/sensor/waterLevel**, and **/sensor/ph** updating roughly every 1.5 seconds, and **/history** gaining a new entry every 5 minutes.

If a sensor misbehaves only once everything else is connected (worked alone, breaks in the full circuit) — that's almost always a shared-ground or power-supply-sag issue, not a code issue: too many things pulling current from the same 5V/3.3V rail.

Part 4 — Note: no soil sensor in this code

The project description mentions a soil sensor alongside pH, TDS, temperature, and water level, but the current code only reads ultrasonic, TDS, thermistor, and pH — there is no soil sensor wiring or reading logic yet. If a soil sensor still needs to be added, it needs its own free analog pin (e.g. GPIO 33 or GPIO 27, both still on the ADC1 group so WiFi won't interfere with it) and a matching new block of code. Flag this for a follow-up pass rather than assuming it's already covered.

Part 5 — Troubleshooting quick reference

Symptom	Likely cause / fix
IDE can't find the board / no COM port appears	Try a different USB cable (some are charge-only); install CP2102 or CH340 USB driver for your OS
Upload fails with a connection timeout	Hold the BOOT button on the board while upload starts, release once "Connecting..." dots appear
Serial Monitor shows garbled/random characters	Baud rate dropdown isn't set to 115200 — match <code>Serial.begin(115200)</code> in the code
WiFi never connects	Wrong SSID/password for the network currently in range, or a 5GHz-only network (ESP32 needs 2.4GHz)
A sensor reads correctly alone but glitches once everything is wired	Missing shared ground somewhere, or too many parts drawing current from one supply rail
Board resets / WiFi drops exactly when the servo or a relay activates	Current spike browning out the ESP32 — move that part's power to an external supply
Temperature reading is wildly wrong (e.g. negative or 300°C)	Thermistor and resistor legs swapped, or a loose connection at GPIO32

AquaGuard IoT capstone project — hardware reference document.